

UNIVERSITÉ DE NEUCHÂTEL

Institut d'informatique

Can LLMs Dream of a Research Community?

*Generation, Peer Review, and Iteration Compared Against
ICLR 2019*

Bachelor Thesis

by

Verica Dimitrova

verica.dimitrova@unine.ch

MATR: 22826721

Supervisor: Prof. Valerio Schiavoni

Bachelor of Science in Data Science

August 2026

Acknowledgements

I express my sincere gratitude to my supervisor, Professor Valerio Schiavoni, for his guidance, availability, and valuable feedback throughout the development of this project. His advice was greatly appreciated and contributed significantly to the completion of this thesis.

Contents

Abstract	4
1 Introduction	5
2 Background and Related Work	5
2.1 LLM Agents	5
2.2 Retrieval-Augmented Generation	6
2.3 LLM-simulated Research Community	6
2.4 Paper plausibility	7
2.5 LLM-as-Judge	7
2.6 ICLR and OpenReview	8
3 System Design and Implementation	9
3.1 ICLR Ingestion	9
3.2 Generation Pipeline	10
3.2.1 Paper-Specific Indexing	10
3.2.2 Frozen Idea Generation	11
3.2.3 Section-by-Section Generation	12
3.2.4 Paper Assembly	13
3.3 Review Pipeline	14
3.3.1 Review-Specific Indexing	14
3.3.2 Multi-Agent Review and Area Chair Decision	14
3.4 Next Iteration	15
4 Simulation Setup	15
4.1 Dataset	15
4.2 Configuration	16
4.2.1 Models	16
4.2.2 Idea Design	17
4.3 Evaluation Metrics	17
4.3.1 Generated Paper Evaluation	18
4.3.2 Review Evaluation	19
4.3.3 Iteration Evaluation	20
5 Results	20
5.1 Generation Pipeline	20
5.2 Review Pipeline	22
5.3 Next Iteration	24

6	Discussion	24
6.1	Generation Pipeline	24
6.2	Review Pipeline	25
6.3	Next Iteration	26
7	Limitations	27
8	Conclusion and Future Work	28
	References	29
A	Database Schema	31
B	Prompt	31
B.1	Topic Expansion Prompt	31
B.2	Abstract Generation Prompt	32
B.3	Section Generation Prompt	32
B.4	Section-Specific Instructions	33
B.5	Rating Scale Guide	35
B.6	Official Review Prompt	35
B.7	Area Chair (Meta-Review) Prompt	37
C	Computational Cost	38

Abstract

This thesis simulates a scientific research community with LLM agents and examines it at every stage of a research cycle, from paper generation through peer review and final decisions to the topics that carry over into the next iteration. In the first iteration, seven open-weight models, from four model families, generated 1,400 papers on 200 topics through a retrieval-grounded pipeline, three reviewer agents produced 4,200 reviews, and an Area Chair made the final accept or reject decisions. Only topics with an accepted paper were carried forward. Real papers and reviews from ICLR 2019, retrieved from OpenReview, served as the human baseline. The generated papers matched real ones in length and sentence structure, but were consistently harder to read, more repetitive, and cited far less, and even the closest model, Qwen3.5-9B, did not match the baseline on every dimension. The reviews read like real ones, but the ratings clustered in a narrow band, and the Area Chair accepted only 14.1% of the papers compared to an acceptance rate of 35.4% at ICLR, while accepted and rejected papers received relatively similar reviewer ratings. The reviewers did not favour their own papers, and across two iterations the topic set narrowed no more than expected by chance. The simulation therefore resembled a real research community mainly on the surface.

1 Introduction

Large language models are already used across many parts of scientific research, from writing assistance to reviewing and literature search. This raises the question of whether these separate uses could be combined into a system that carries out the research process on its own, without human involvement. Answering this requires more than checking whether a model can write a plausible paper. In a real research community, papers are judged by reviewers, only a small share is accepted, and the accepted work shapes what is studied next. A simulated community therefore has to be examined at every one of these stages. These considerations lead to the following research questions:

1. Can LLMs generate papers that are structurally and stylistically close to real ones, and which models come closest?
2. Can they review these papers in a way that resembles real peer review?
3. Do they evaluate reliably, or do their ratings cluster and favour their own work?
4. When only topics linked to accepted papers carry over into the next iteration, does the topic set narrow, and is that narrowing caused by differences between topics or explainable by chance?

This thesis builds a simulation to answer these questions and compares it against a real ICLR conference, used as a human baseline for both the generated papers and the generated reviews.

Three contributions were made in this thesis. The first is the simulation system itself, which consists of an ingestion pipeline for real ICLR data, a retrieval-grounded generation pipeline, a review pipeline with three reviewer agents and an area chair, and an iteration step that carries accepted topics forward. The second is an empirical comparison of this community against a real one, covering 200 topics, 1,400 generated papers and 4,200 reviews in the first iteration, followed by a second iteration. The third is an analysis addressing the research questions above and evaluating the behaviour of the simulated community across its different stages.

2 Background and Related Work

2.1 LLM Agents

A large language model, on its own, generates a single response to a prompt and has no way to remember what happened before or to act beyond producing text. An LLM agent builds on top of this by giving the model a defined role, a memory, and the ability to plan a sequence of actions toward a goal. Wang et al. (2024) describe an LLM agent as

composed of four core modules, profiling, memory, planning, and action, that together let the model take on a persona and pursue a task over multiple steps.

Every agent is powered by a language model, and the choice of model family shapes how the agent behaves in terms of instruction following, reasoning, output generation, and computational requirements. This thesis uses four open-weight families, Llama, developed by Meta (Grattafiori et al., 2024); Qwen, developed by Alibaba (Yang et al., 2024, 2025); Phi-4, developed by Microsoft (Abdin et al., 2025); and a recent Swiss model, Aper-tus (Project Apertus, 2025), developed by EPFL, ETH Zurich, and the Swiss National Supercomputing Centre.

2.2 Retrieval-Augmented Generation

One of the main limitations of language models is that everything they produce comes only from what was learned during training, without direct access to external reference material at generation time. This becomes especially important in tasks like writing and reviewing research papers. For this thesis, an approach called Retrieval-Augmented Generation (RAG) was used to help tie the generated output to real academic writing and peer-review language by retrieving relevant material directly from ICLR papers and reviews. This approach was introduced by Lewis et al. (2020), who showed that combining a language model with a retrieval step produces output that is more specific and factually grounded than generating from the model’s parameters alone.

2.3 LLM-simulated Research Community

An LLM research community is a system where several language model agents take on roles which are normally filled by people in a research group. Instead of one model doing everything, one agent generates a research idea, another writes it up as a paper, and a third reviews it, creating a multi-agent system. The agents interact with each other in a way that real researchers, authors, and reviewers do during a real research cycle.

There are several recent works that explored building such systems end to end. The AI Scientist (Lu et al., 2024) is a fully automated pipeline that generates a research idea, writes and runs the code needed to test it, and writes up the results as a paper. The same system also includes an automated reviewer evaluated against 500 real ICLR 2022 papers. CycleResearcher (Weng et al., 2024) follows a similar idea, but post-trains open language models on real papers and reviews. It combines a CycleResearcher model for research planning and paper generation with a CycleReviewer model for peer review, whose feedback is then used to iteratively improve the researcher through a research-review loop.

A separate line of work looks specifically at the review side of this process. ReviewA-gents (Gao et al., 2025) uses several independent reviewer agents that each read a paper

and produce their own review through a staged process of summarizing the paper, analyzing it, and reaching a conclusion. This is followed by a separate meta-reviewer agent that reads these reviews and produces a final decision.

A separate concern applies specifically to running such systems iteratively. Shumailov et al. (2024) describe this risk as model collapse, a degenerative process in which a model trained on data produced by earlier generations gradually loses information about the original distribution it started from. One of the mechanisms behind this is statistical approximation error and occurs simply because each generation only has a finite number of samples to take from, so rare or low-probability outcomes can disappear purely by chance.

2.4 Paper plausibility

Writing style can reveal a lot about how a text was produced, even before its content is judged. Evaluating whether generated text is stylistically and structurally plausible has become a growing concern as large language models are increasingly used to produce writing that resembles human work. Chen et al. (2026) show this directly, comparing human and ChatGPT-written essays and finding that AI-generated text is stylistically far more uniform in its word usage, while human writing varies much more from one piece to another. This kind of stylistic contrast is not limited to word usage. Shumailov et al. (2024) observed that generated text also tends to contain a large number of repeating phrases across nearly all text-generation models. An older study by Flesch (1948) introduced a score that captures how easily a text can be read from its sentence length and syllable count, which offers another dimension along which generated and real writing can be compared.

2.5 LLM-as-Judge

For a long time, evaluating generated text automatically has been a difficult task, and even traditional metrics such as BLEU and ROUGE could not come close to real human judgment on open ended tasks. This motivated the LLM-as-judge approach, which uses large language models themselves to evaluate generated text.

G-Eval (Liu et al., 2023) is one of the earlier frameworks built around this idea. It has an LLM generate its own evaluation steps as a chain-of-thought, then fill out a scoring form for each quality dimension. This reaches a much higher correlation with human ratings than prior metrics. The authors also report a limitation that raw scores tend to cluster on a single value with little variance, which they address by weighting scores by their output probabilities.

This clustering problem turned out to be only one of several issues with using LLMs as judges. Zheng et al. (2023) coined the term LLM-as-a-Judge and studied its failure

modes more directly. They found that LLM judges are likely to show position bias, favoring whichever answer appears first, verbosity bias, favoring longer answers, and self-enhancement bias, favoring answers from the same model family as the judge. They introduced MT-Bench and Chatbot Arena to measure these effects and proposed mitigations such as swapping answer order and using few-shot examples.

Given these challenges, later work has looked at how evaluation design can improve the reliability of LLM judges. As part of a broader benchmark for evaluating deep research agents, ResearchRubrics (Sharma et al., 2025) also examines how the design of the evaluation prompt affects an LLM judge’s reliability. In an ablation study, adding concrete examples to each rubric criterion improved agreement with human evaluators by 3-4% for binary rubrics, while having a separate LLM automatically expand and rephrase the same criteria degraded agreement by 15-20%. Based on these results, the authors concluded that precise rubrics work better than overly detailed ones.

2.6 ICLR and OpenReview

The International Conference on Learning Representations (ICLR) is, together with NeurIPS and ICML, one of the three highest-impact venues in machine learning research. It was founded by Yann LeCun and Yoshua Bengio in 2012 and held its first edition in 2013 in Scottsdale, Arizona. What distinguishes ICLR from most other venues, and what makes it particularly useful for this thesis, is that it has used an open peer review process since its founding, based on a model proposed by LeCun as an alternative to traditional models that are fully anonymous with closed conference reviewing (LeCun, 2009). Under this model, submitted papers, their reviews, and the discussions between authors and reviewers are made publicly visible. This openness is what makes it possible to obtain, for a very large number of papers, not only the accept or reject outcome but also the full text of the reviews that led to it, which made ICLR an adequate human baseline for this thesis.

This open review process is established through OpenReview.net, a nonprofit peer review platform founded in 2013 by Andrew McCallum’s research group at the University of Massachusetts Amherst (OpenReview, n.d.-a). OpenReview has become the standard reviewing infrastructure for ICLR and several other major machine learning conferences and exposes its data through a public API, which allows submissions, reviews, and decisions to be retrieved programmatically.

Several OpenReview-specific terms are used throughout this thesis to describe how the data in a conference is organized. A venue refers to a specific conference and year hosted on OpenReview, such as ICLR 2019. In each venue, papers are submitted through a submission invitation, which defines the submission form. Each submission is stored as a top-level note and creates its own forum, where the paper’s review process is recorded. Official reviews, meta-reviews, decisions, author responses, and comments are stored as

reply notes within this forum (OpenReview, n.d.-b). Authors also provide keywords during submission to describe the main topics of their paper.

3 System Design and Implementation

To investigate the overall behavior of the simulated LLM-based research community, two main pipelines were designed: a generation pipeline and a review pipeline. This separation reflects the division of roles found in real research communities, where paper generation and peer review are treated as distinct processes handled by different people.

The generation pipeline’s main focus was to retrieve and embed relevant information from real ICLR papers, generate ideas, generate sections for each idea, and assemble them into a single structured paper. The review pipeline, on the other hand, focused on reviewing these generated papers and producing structured reviews, ratings, and final decisions. The iteration loop served as a bridge between the two pipelines, capturing the transition of the simulation from one iteration to the next.

Both pipelines used real ICLR data as a human baseline, though for different purposes. The generation pipeline relied on real ICLR papers, while the review pipeline relied on real ICLR reviews. Section 3.1 describes how this data was collected and prepared for both pipelines.

3.1 ICLR Ingestion

The human baseline was obtained through `iclr_general_ingest.py`, which was designed to support multiple ICLR conference years. The script retrieved conference data directly from the OpenReview API and normalized it into a relational SQLite database.

For a given ICLR conference year, the script first constructed the expected conference venue identifier, while allowing it to be manually overridden when necessary, and detected whether that venue used OpenReview API v1 or v2. Instead of assuming the API version from the conference year, the script first checked whether the venue was available through API v2 and switched to API v1 when necessary. It then resolved the submission invitation. Since OpenReview’s invitation naming has changed across conference years, the script checked several common invitation names, including `Blind_Submission`, `Post_Submission`, `Submission`, and `Paper`, and searched for another suitable invitation if none of them matched. Desk-rejected and withdrawn submissions were excluded, since they do not represent papers that completed the standard review process.

Once the submission invitation had been resolved, the script retrieved every relevant paper returned through that invitation and attempted to retrieve all the notes attached to each paper’s discussion forum. Each retrieved note, based on the note’s invitation identifier, was classified according to its role in the conference review process, such as

official review, meta-review, decision, author response, or general comment.

An important element for the simulation was the PDF text extraction, which was treated as a separate step in the ingestion process. For each paper with a PDF URL, the script first resolved the URL to an absolute address, then downloaded the PDF over HTTP. If the download failed, the script retried up to five times with increasing wait times between attempts. Once downloaded, the PDF’s text was extracted page by page using PyMuPDF and concatenated into a single plain-text document. If the extracted text was less than a minimum length, the paper was marked as having no usable text. This case was expected mainly for scanned or image-based PDFs with no embedded text layer to extract.

The resulting database stored each retrieved paper and note using a two-layer approach, where every record was kept in full as raw JSON, together with a set of normalized fields extracted from it. Each paper record included its title, abstract, PDF URL, and timestamps, and each note record included its type, forum, invitation, and its parent note. For notes classified as official reviews, an additional normalization step checked a list of candidate field names, such as `rating` versus `recommendation` versus `overall_recommendation`, and stored the matching value together with confidence, summary, review text, strengths, weaknesses, and questions, when present, into the `reviews` table. Final decisions were normalized in a similar way and stored in the `paper_decisions` table. Notes explicitly typed as decisions were stored directly. Notes that were not explicitly typed as such but still contained decision related content, such as a recommendation paired with a meta-review comment, were also treated as decisions, since some editions did not tag the decision note with a distinct type. The complete database schema, including the remaining tables, is provided in Appendix A.

3.2 Generation Pipeline

3.2.1 Paper-Specific Indexing

The data ingested from the ICLR conference and stored into the SQLite database was prepared for retrieval by the generation pipeline through `embedding_index.py`. The purpose of this retrieval step was to provide the generation models with relevant examples from real ICLR papers when producing each section of a generated paper.

In this script, papers were selected from the SQLite database, loaded if they had a non-empty abstract, and prioritized by whether their PDF text had been successfully extracted. Papers from ICLR conferences do not follow a completely uniform structure since section organization and naming often depend on the authors’ writing styles and document formatting. Despite this variability, the script attempted to clean the text and split it into sections by matching common section headers, such as introduction, related work, method, and experiments, and normalized header variants, such as methodology,

approach, and model, into a smaller set of canonical section names. When successfully detected, sections corresponding to the abstract, references, appendix, and front matter were discarded before chunking, since the abstract was already indexed separately and the remaining sections were considered noisy elements of the text that could introduce irrelevant content into retrieval results.

Within each constructed section, text was grouped into paragraph-based chunks targeting approximately 1200 characters. The chunk size was chosen to be big enough to keep each chunk focused on a specific idea and to preserve coherent context between sentences. Overlap between consecutive chunks was handled at the paragraph level so the last paragraph of each chunk was carried over as the start of the next. Each resulting chunk was labeled with its chunk type, such as an abstract chunk, a detected section chunk, or a full-text PDF-body chunk, and linked back to its source paper and section name. When no section headers could be detected at all, the script instead sliced the cleaned text into fixed-size windows of 1200 characters, with an overlap of 200 characters between each consecutive window.

Chunks were embedded in batches using the `nomic-embed-text` model served locally through Ollama. The resulting vectors were stored in a ChromaDB collection, together with the original chunk text and metadata identifying each chunk’s source paper, title, chunk type, and section name. This structure later allowed the generation pipeline to perform cosine similarity search and to filter retrieval results by paper, chunk type, or section.

3.2.2 Frozen Idea Generation

Each paper submitted to the ICLR conference emerges from an idea that motivates its research direction and shapes the main question addressed in the paper. `generate_frozen_ideas.py`, together with generation functions defined in `generate.py`, incorporated this same principle into the generation pipeline by producing these ideas and freezing them afterwards, creating a controlled environment for model comparison.

The list of research topics was initially derived from real ICLR paper metadata. The script queried the ingested database for each paper’s OpenReview keyword field, counted how frequently each normalized keyword occurred, and merged synonymous variants and expanded abbreviations, such as `gan` into "generative adversarial networks", and `cnn` into "convolutional neural networks". Keywords that were considered too generic, such as deep learning or training, were filtered out before ranking the remaining keywords by frequency. These ranked keywords were then passed to an LLM, which rewrote them into more precise and diverse research topic phrases instead of leaving them as raw keyword strings, providing more context for the next steps. The generated topic phrases were cleaned, deduplicated, and saved to a topics file so that the same topic list could be reused in later runs.

For each topic, the generation functions in `generate.py` selected a research direction from a fixed list of broad research areas taken from the ICLR 2019 Call for Papers (International Conference on Learning Representations, n.d.), refined it into a concrete problem statement, and generated a set of candidate titles. For each candidate title, a separate abstract was generated, creating frozen ideas that shared the same direction and refined problem but differed in title and abstract. Topics were processed in batches, where each topic was assigned a sequential base identifier and each candidate idea was assigned an additional numerical suffix. The final frozen ideas were saved as individual JSON files for later reuse by the generation pipeline.

The generation functions used throughout this stage were part of `generate.py` which is a shared module that also implements the retrieval and section-generation logic described in the next subsection.

3.2.3 Section-by-Section Generation

Building on the frozen ideas produced in the previous stage, `generate_sections_from_ideas.py` orchestrated the generation of each paper’s six sections: introduction, related work, background, method, experiments, and conclusion. Instead of generating one full paper, the script generated each section separately. Generating the complete paper in a single pass would have made it harder for the models to maintain coherence over the required length. These six sections were chosen to closely resemble the structure of real ICLR papers, making the generated papers realistic enough for a meaningful comparison.

Before section generation began, the full set of frozen ideas was shuffled once using a fixed random seed and then split into equally sized, non-overlapping blocks. Each block was assigned to a model that was included in the run, so that each model generated papers from its own distinct subset of ideas.

For each target section, retrieval was performed based on a predefined mapping between that section and related source-section types. Table 1 shows the mapping for all the sections. The retrieval step initially retrieved a larger set of candidate chunks, removed near duplicate text, and limited how many chunks could come from any single source paper, ensuring that the final retrieved context was taken from a diverse set of papers instead of being dominated by one. This context was then combined with the frozen idea’s topic, direction, refined problem, title, and abstract into a section-specific prompt. Each target section had its own instructions specifying an approximate paragraph count and target length, together with constraints such as not fabricating citations or experimental numbers not supported by the abstract. The retrieved context was meant only as inspiration for style, terminology, and structure. The prompt instructed the model not to copy or adapt a retrieved paper’s method as its own and to stay within the frozen idea it had been assigned. The generation call itself was sent to either a locally served Ollama model or a Hugging Face model, depending on the model identifier. Because Hugging

Face models ran on different infrastructure than Ollama models, they were generated as separate batches. For that reason, the script included an offset parameter that allowed an idea assignment to resume from a later point in the shuffled set. This allowed the Hugging Face model to run after the other models had generated their sections without reusing ideas that had already been assigned.

The script was designed so that section generation could be run for the complete set of sections in order or for a chosen subset. If generation failed for a particular idea, model, and section combination, the error was logged and the batch continued with the next job instead of stopping entirely. Each successful generation was saved as a JSON file and named by combining the idea identifier and section name. This file contained generated text, together with the idea metadata, retrieved context, token usage, and timing statistics.

Target section	Retrieved source sections
Introduction	Abstract, Introduction, Related Work, Background
Related Work	Related Work, Background, Introduction
Background	Background, Introduction, Related Work
Method	Abstract, Background, Method
Experiments	Method, Experiments, Conclusion
Conclusion	Abstract, Experiments, Conclusion

Table 1: Mapping between generated paper sections and the source-section types used for retrieval

3.2.4 Paper Assembly

The final step of the generation pipeline was handled by `assemble_papers.py`, which combined the individually generated sections into complete paper files. For each model and frozen idea, the script collected all of its generated section files and merged them into a single paper JSON, saved separately per model.

For each of the six target sections, the script checked whether a generated file existed. If it did, the generated text was added to the assembled paper. If not, the section was recorded as missing. This allowed papers with incomplete sections, for example due to a failed generation job in the previous stage, to still be assembled, with the missing sections explicitly listed alongside the ones that were successfully generated. Each final JSON file contained its metadata, its available sections, and a list of any missing sections.

3.3 Review Pipeline

3.3.1 Review-Specific Indexing

To support the review pipeline, `embedding_index_review.py` built a separate retrieval index from real ICLR reviews already stored in the database produced by the ingestion. Its purpose was to give the review pipeline access to real review language and structure, so that generated reviews could be calibrated in style. For each review, the script combined the summary, strengths, weaknesses, questions, and free text review fields into a single labeled block.

Reviews shorter than 200 characters were excluded, since they contained too little text to provide a useful review example. Reviews exceeding 6000 characters were truncated at the nearest sentence boundary where possible, to keep the length of the indexed reviews more consistent. Because each review was already short enough, it was embedded as a single document instead of being split into chunks, using the same `nomic-embed-text` model as in the generation pipeline. The resulting embedding was stored in a separate ChromaDB collection dedicated to reviews, together with metadata including the associated paper, rating, and confidence score.

3.3.2 Multi-Agent Review and Area Chair Decision

The core of the review pipeline was implemented in `review.py`, which reviewed each paper produced in the generation pipeline.

For each paper, three independent reviewer models retrieved the most similar reviews from the specific review index described in the previous section and each produced one official review in a single generation call. This number matches the average number of reviewers observed in the real ICLR dataset used as the human baseline (Section 4.1), and is also consistent with the ablation study in ReviewAgents (Gao et al., 2025), which found that three reviewers produced the closest alignment with human feedback.

The prompt instructed each reviewer to write summary, strengths, weaknesses, questions and recommendation rationale sections inside a single review field. The rating scale with its meaning was given to each reviewer as an explicit reference. The prompt also included two calibration examples showing how specific evidentiary gaps, such as missing baseline comparisons or no engagement with the closest prior work, should map onto specific rating ranges, together with a reminder that most real submissions are rejected and that the scale genuinely extends down to 1–2 when necessary. Retrieved reviews were framed strictly as stylistic inspiration, with instructions not to copy or reference them directly. The output format was designed to follow the structure used in real ICLR 2019 reviews on OpenReview.

Once all three official reviews for a paper were produced, a single Area Chair model read only these reviews, not the paper itself, and produced a final accept/reject decision

together with a justification. A second prompt was constructed for this separate role, which required the Area Chair to work through an explicit procedure containing three steps. The Area Chair first restated the three ratings and confidences, then counted how many of the three reviews raised at least one significant weakness (serious concerns about the paper’s central claims, novelty, or evidence), and finally judged whether at least two independent reviewers identified the same underlying weakness or whether their weaknesses were scattered across different points. More convergence toward the same weakness encouraged the Area Chair to lean toward rejection. Three calibration examples illustrated cases with no significant weaknesses, convergent weaknesses, and scattered weaknesses. At the end, the Area Chair reported a summary, consensus strengths and weaknesses, this significant weakness count, a final decision, and a justification explicitly stating whether the weaknesses converged or were scattered. The overall reasoning structure of both the reviewer and Area Chair prompts was inspired by the staged Summarize-Analyze-Conclude process used in ReviewAgents (Gao et al., 2025). The full prompt templates used throughout the generation and review pipelines are provided in Appendix B.

Several versions of the reviewer and Area Chair prompts were tested before the final version was chosen. In particular, early versions of the Area Chair prompt resulted in almost all papers being rejected, while adding concrete calibration examples produced a less extreme decision distribution. The use of concrete examples was inspired by ResearchRubrics (Sharma et al., 2025), which found that adding examples to rubric criteria improved agreement between LLM judges and human evaluators.

3.4 Next Iteration

Between iterations, `prepare_next_iteration.py` selected a subset of the topics linked to accepted papers in the previous round and prepared them for a new generation pass. The selected topics were passed back into the frozen idea generation stage of the generation pipeline, producing a new set of frozen ideas that were processed through the same generation and review pipelines.

4 Simulation Setup

4.1 Dataset

The human baseline used throughout this thesis was taken from a single year of the ICLR conference, ICLR 2019, ingested through the pipeline described in Section 3.1.

The resulting SQLite database contained 1,418 papers with successfully extracted PDF text and final decisions. Papers explicitly marked as desk-rejected or withdrawn were excluded during ingestion and are therefore not part of this count. Out of the 1,418 papers, 916 were rejected and 502 were accepted, showing an acceptance rate of 35.4%.

Each paper received a mean of 3.05 official reviews (median 3), for a total of 4,329 official reviews.

Mean review ratings differed considerably between the two decision groups: rejected papers received a mean rating of 4.80 (median 5.00) against a mean of 6.61 (median 6.67) for accepted papers, while mean reviewer confidence was comparable across groups (3.79 for Reject, 3.70 for Accept).

Paper length was estimated from the cleaned PDF text after removing back matter (references, appendices, acknowledgments). Abstracts averaged 162 words (std 45.2). The estimated main-body length averaged 4,668 words (std 771.9) and ranged between 1599 and 8,995 words across the full corpus, with accepted papers running slightly longer on average (mean 4,951.4 words) than rejected papers (mean 4,512.2 words).

The papers were also described using several writing style metrics. Each paper contained on average 223 sentences, with a mean sentence length of 21.2 words (median 19.0). Lexical diversity, repetition, and citation count were also measured, using the metrics defined in Section 4.3.1. Type-token ratio (TTR) averaged 0.296 (std 0.028), trigram repetition (rep3) averaged 0.113 (std 0.035), and citation count averaged 14.46 per paper (std 10.59), with 89.3% of papers containing at least one citation. Readability was measured with the Flesch Reading Ease score and averaged 38.8, placing the papers in the “Difficult” band (Flesch, 1948, Table 5).

4.2 Configuration

4.2.1 Models

In total, seven language models were chosen for this research community simulation: Llama-3.1-8B, Qwen2.5-7B, Qwen3-8B, Qwen3-14B, Qwen3.5-9B, Phi-4-reasoning-14B, and Apertus-8B-Instruct-2509. The models come from four different families and differ in size from 7 billion to 14 billion parameters. They were selected primarily because of their diversity across model families, while the size range was kept narrow to make the comparison between models more balanced. A temperature of 0.7 was used across all generation models to allow the models to produce varied text and to avoid fully deterministic outputs.

Apertus-8B was the only model served through the Hugging Face infrastructure, while the six remaining models were served locally through Ollama. Apertus-8B exists on Ollama, but its version was provided by a third party, and some functionalities were reported as unsupported (Ollama, 2025), so a more reliable choice was to use it directly from Hugging Face.

In the generation pipeline (Section 3.2), frozen ideas were generated using a single model, Qwen3-14B. This model was chosen on the assumption that a larger model would produce more coherent and diverse research ideas than the smaller generators. In contrast,

all seven models were used for paper generation to introduce variation in writing style and to compare how closely each model matched the human baseline.

For the review pipeline (Section 3.3), only a subset of these models was used to follow the peer-review logic described in ReviewAgents (Gao et al., 2025). Llama-3.1-8B, Qwen3-14B, and Phi-4-reasoning-14B were chosen as the three independent reviewers, while Phi-4-reasoning-14B was also assigned the Area Chair role. These three models also generated papers, which means that each one occasionally reviewed papers it had itself written. This overlap is what made it possible to analyze self-preference later in the evaluation.

Alongside the seven generator models, a single embedding model, `nomic-embed-text`, was used throughout the retrieval components of the simulation and served locally through Ollama. It was used for indexing in both the generation and review pipelines, and later to embed reviewer weaknesses for the reviewer analysis.

The resulting token usage and runtime are reported in Appendix C.

4.2.2 Idea Design

The frozen ideas used in the first iteration of the simulation were generated from the 200 highest ranked OpenReview keywords, which were transformed into 200 research topics as described in Section 3.2.2. For each topic, seven candidate titles and matching abstracts were generated, producing $200 \times 7 = 1,400$ distinct frozen ideas. All 1,400 ideas were shuffled using a fixed seed of 42 and split into seven blocks of 200 ideas each, with each block assigned to one of the seven generator models. The total of 1,400 generated papers was chosen to remain close to the 1,418 papers in the ICLR 2019 human baseline. Each generated paper was later reviewed by three independent reviewer agents, producing 4,200 reviews in total.

Idea generation for the second iteration followed the same procedure, except for the source of its topics. Instead of using OpenReview keywords, the second iteration used topics that had produced at least one accepted paper in the first iteration. Because of the time and computational constraints of generating and reviewing another full iteration, only 40 topics were kept for the second iteration. These 40 topics were passed through the same idea-generation step as before, again producing seven candidate titles and matching abstracts per topic, for a total of $40 \times 7 = 280$ frozen ideas. These ideas were then processed through the same generation and review pipelines as in the first iteration, producing 280 papers and 840 reviews.

4.3 Evaluation Metrics

The two pipelines, together with the transition to the next iteration, addressed different questions and were evaluated with different sets of metrics.

4.3.1 Generated Paper Evaluation

The generation pipeline was evaluated by measuring how closely the generated papers matched the ICLR 2019 human baseline in structure and writing style, and by comparing how close each of the seven generator models came to that baseline.

Each generated paper was characterized by its total word count and mean sentence length. Lexical diversity was also measured with the type-token ratio (TTR), defined as

$$\text{TTR} = \frac{\text{number of unique words}}{\text{total number of words}}$$

and computed over the first 2,000 words of each paper so that the value remained comparable across papers of different lengths. Repetition (rep3) was measured as the fraction of repeated word trigrams, sequences of three consecutive words:

$$\text{rep3} = 1 - \frac{\text{number of unique trigrams}}{\text{total number of trigrams}}.$$

A low TTR or a high repetition fraction indicates repetitive vocabulary or looping text.

The number of citations used in the text of the papers was counted using patterns that match both the bracketed-numeric (e.g. [12]) and the author-year (e.g. (Author, 2019)) citation styles. Generation prompts specifically forbade fabricating citations, so this metric captures whether the models nonetheless produced text that resembles scientific writing, and how its citation count compares to that of real papers. Additionally, a random sample of 20 citation instances (seed 42) was manually checked to see whether they cited existing literature.

Text readability was measured with the Flesch Reading Ease score (Flesch, 1948), which is defined as

$$206.835 - 1.015 \times \frac{\text{words}}{\text{sentences}} - 84.6 \times \frac{\text{syllables}}{\text{words}}$$

where scores are separated in seven bands ranging from “Very difficult” (0–30) to “Very easy” (90–100) (Flesch, 1948, Table 5).

To test whether the differences observed between models were statistically significant, a Kruskal-Wallis H-test was applied to four metrics (paper length, TTR, rep3 and Flesch) across the seven generator models. The non-parametric test was chosen because several metric distributions were skewed and contained outliers, violating the assumptions of an ANOVA. A p-value below 0.05 indicates that at least one model’s distribution differs significantly from the others for that metric.

A closeness score was computed as the mean absolute standardised difference between each model and the ICLR baseline across the six metrics. Each difference was divided by the corresponding ICLR standard deviation, so that metrics with different scales could

be combined. Lower scores indicate greater similarity to the human baseline.

4.3.2 Review Evaluation

The review pipeline was evaluated by comparing its metrics with the corresponding statistics from real ICLR reviews, and by examining evaluator reliability and potential biases. Together, these metrics address two separate research questions in this thesis.

Peer-review resemblance. Four metrics were used. First, the rating distribution captured the mean and standard deviation of ratings given by each reviewer model.

Second, inter-reviewer agreement was measured as the standard deviation between the three reviewers’ ratings per paper, along with the share of papers where all three agreed within one point.

Third, weakness-category analysis matched each generated review’s extracted weaknesses against nine fixed keyword-based categories (e.g., missing baselines, novelty concerns, limited datasets, lack of ablations). The same categories were applied to real ICLR review text, and their frequencies were then compared.

Finally, the Area Chair’s decision versus reviewer ratings examined the behaviour of the meta-reviewer. The accept rate of the pipeline was computed, and the mean ratings of accepted and rejected papers were compared.

Evaluator reliability and potential biases. The same rating data was also examined per reviewer model to check for score clustering, that is, whether a model repeatedly assigned the same one or two ratings regardless of the paper.

Since three reviewer models were used as both paper generators and reviewers, generator self-preference was tested to measure whether a model favoured the papers it authored over those authored by other models. A Welch t-test was used to check whether this difference was significant. Additionally, Area Chair self-alignment was tested in a similar way, by checking whether the Area Chair’s final decision correlated more strongly with the rating given by the reviewer sharing its model than with the mean rating of the other two reviewers. A paired bootstrap test was used to determine whether the difference between these correlations was statistically significant.

The final metric examined reviewer convergence in relation to the Area Chair’s final decision. The extracted weaknesses from each paper’s three reviews were embedded using the `nomic-embed-text` model, and their mean pairwise cosine similarity was used as a measure of reviewer convergence. A Mann-Whitney U test was then used to test whether rejected papers showed greater reviewer convergence than accepted papers.

4.3.3 Iteration Evaluation

The iteration evaluation examined whether keeping only topics linked to accepted papers caused the set of topics to narrow more than expected by chance. The iteration process was evaluated using the topic elimination rate, defined as the proportion of topics with no accepted paper among their seven submissions. This observed rate was then compared with a chance-based null model, where each paper was assumed to be accepted independently according to the overall acceptance rate of its generator model. The expected elimination rate under this model was compared with the observed rate to evaluate whether topic quality influenced the narrowing or whether it could be explained by chance alone.

5 Results

5.1 Generation Pipeline

The structural and stylistic evaluation metrics for the 1,399 generated papers are shown in Table 2 across the seven generator models and the ICLR 2019 human baseline. One Llama-3.1-8B generation was excluded from these metrics since it failed to terminate and produced 262,413 words, which skewed the reported averages. Table 3 reports the overall closeness score for each model.

Across all models, the generated papers had on average 5,762 words (median 5,875), but length varied considerably by model. Qwen2.5-7B produced the shortest papers on average (2,791 words), while Phi-4-reasoning-14B produced the longest (11,465). Compared with the ICLR main-body baseline of 4,668 words, Apertus-8B came closest at 3,990 words. Qwen2.5-7B and Llama-3.1-8B produced shorter papers, the Qwen3 models generally produced longer papers, while Phi-4-reasoning-14B produced papers more than double the human length.

The spread of sentence length was relatively narrow across models. Apertus-8B, with a mean of 21.06 words per sentence, was the closest to the ICLR baseline of 21.2, followed by Phi-4-reasoning-14B at 20.36, while Qwen3-14B wrote the longest sentences at 26.2.

The models also showed considerable differences in writing style. Lexical diversity, measured by type–token ratio, ranged from 0.224 for Apertus-8B to 0.313 for Qwen3.5-9B. Compared to the ICLR baseline of 0.296, Qwen3.5-9B, Qwen3-14B and Phi-4-reasoning-14B were the closest to the human value while Apertus-8B and Llama-3.1-8B used the least varied vocabulary. The trigram-repetition metric showed that every model exceeded the ICLR baseline of 0.113, ranging from 0.174 for Qwen3.5-9B to 0.320 for Llama-3.1-8B. Even the least repetitive generated papers were therefore more repetitive than the real ones.

The citation metric showed that generated papers included far fewer citations than

Model	Words	Sent. length	TTR	rep3	Citations	Flesch
ICLR 2019 (baseline)	4,668	21.17	0.296	0.113	14.46	38.84
Qwen3.5-9B	6,188	23.05	0.313	0.174	0.07	1.46
Qwen3-14B	5,991	26.15	0.288	0.202	1.62	−1.83
Qwen2.5-7B	2,791	22.39	0.256	0.218	4.14	1.40
Apertus-8B-Instruct-2509	3,990	21.06	0.224	0.311	6.12	7.49
Llama-3.1-8B	3,051	22.43	0.229	0.320	4.20	3.17
Qwen3-8B	6,845	24.94	0.273	0.245	3.18	−0.50
Phi-4-reasoning-14B	11,465	20.36	0.285	0.224	0.44	9.00

Table 2: Generated-paper metrics compared with the ICLR 2019 baseline.

the human baseline of 14.5 per paper. The most citation-dense model, Apertus-8B, had an average of 6.12 citations and contained at least one citation in 92% of its papers, yet this was still 58% below the human baseline, while Phi-4-reasoning-14B and Qwen3.5-9B produced almost none (0.44 and 0.07 on average). In total, 3,949 citation instances were extracted across 789 papers (56.4% of the corpus). The manual inspection showed that some citations referred to existing literature, while others combined correct author names and years with either the wrong paper or the wrong technique.

Readability diverged the furthest. On the Flesch Reading Ease scale, every model scored far below the real-paper average of 38.8. Per-model scores ranged from 9.0 for Phi-4-reasoning-14B down to negative values for Qwen3-8B (−0.50) and Qwen3-14B (−1.83). The models with positive scores were placed in the “Very Difficult” band (0–30), below the “Difficult” band that contains the human baseline, while the models with negative scores were below the defined range of the scale altogether.

These per-model differences were statistically significant. For all four metrics tested, paper length ($H = 1323.1$), TTR ($H = 1146.2$), repetition ($H = 1076$) and Flesch score ($H = 584$), the Kruskal–Wallis H-test rejected the null hypothesis of a shared distribution at the 5% level, with $p < 10^{-120}$ in every case.

The closeness score placed Qwen3.5-9B closest overall to the ICLR papers with a score of 2.03, while Phi-4-reasoning-14B was the furthest at 3.10. The remaining models landed between these two, as shown in Table 3. For every model, the distance was dominated by readability and repetition, while sentence length and, for several models, lexical diversity stayed close to the human baseline. Even though Qwen3.5-9B was the closest overall, no model matched the baseline on every dimension.

Model	Closeness score
Qwen3.5-9B	2.03
Qwen3-14B	2.32
Qwen2.5-7B	2.35
Apertus-8B-Instruct-2509	2.46
Qwen3-8B	2.67
Llama-3.1-8B	2.90
Phi-4-reasoning-14B	3.10

Table 3: Closeness to the ICLR 2019 baseline.

5.2 Review Pipeline

The results show that across all 4,200 individual reviews, ratings ranged from 1 to 9, with a mean of 6.21 (std 0.81). Compared to the real ICLR baseline of 5.44 (std 1.49), the generated ratings were both higher and considerably less spread out. Figure 1 shows this difference in the rating distributions.

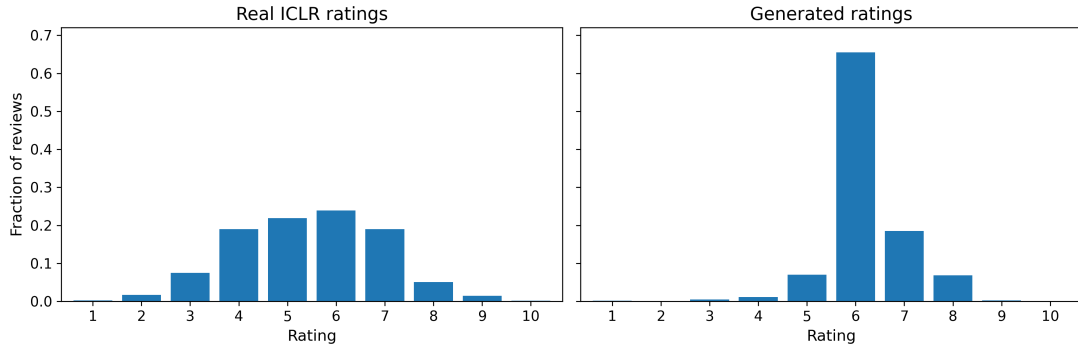


Figure 1: Rating distributions for generated and real ICLR reviews.

Inter-reviewer agreement was high. The standard deviation between the three reviewers’ ratings on the same paper averaged 0.52 (median 0.47), and in 68.9% of papers all three reviewers agreed within one point of each other.

The weakness categories raised by the reviewers diverged from those raised in real ICLR reviews. Missing baselines was the most common category in both the generated (47.9%) and real reviews (61.3%), and novelty concerns were also common in both (30.7% vs 43.4%). The largest gaps were in limited dataset concerns, raised in 30.3% of generated reviews against 3.0% of real reviews, and scalability concerns, raised in 28.0% against 4.9%. Theoretical gap concerns were raised less often by the generated reviewers than by real ones (12.3% vs 26.2%). Figure 2 shows the full comparison across categories.

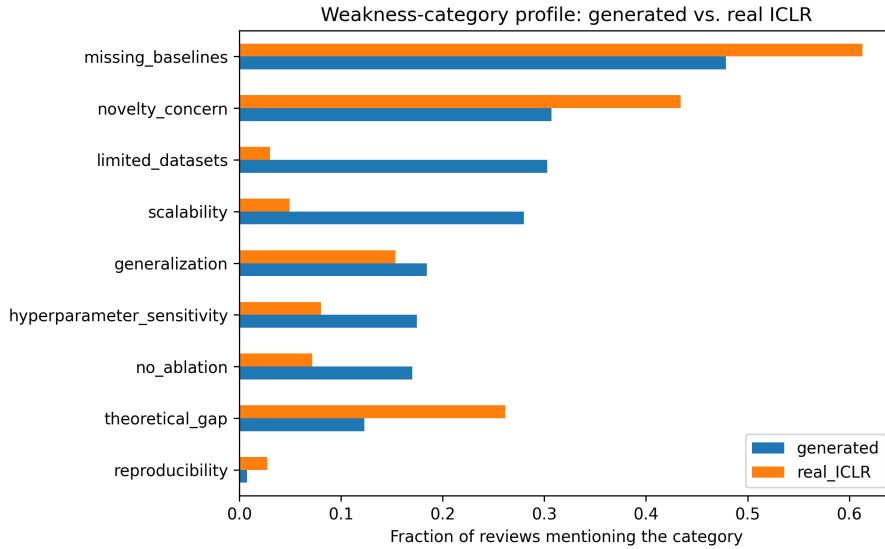


Figure 2: Weakness-category frequencies in generated and real ICLR reviews.

Of the 1,400 papers, 198 (14.1%) were accepted by the Area Chair. Accepted papers had a mean rating of 6.51 (std 0.45) and rejected papers had a mean rating of 6.16 (std 0.42). The review pipeline’s acceptance rate of 14.1% was well below the real ICLR accept rate of 35.4% (502 / 1,418), meaning that the pipeline was considerably more conservative in its final decisions, despite giving individual ratings that were on average higher than those of real reviewers. Figure 3 shows the progression from generator model to mean rating band and final accept/reject decision.

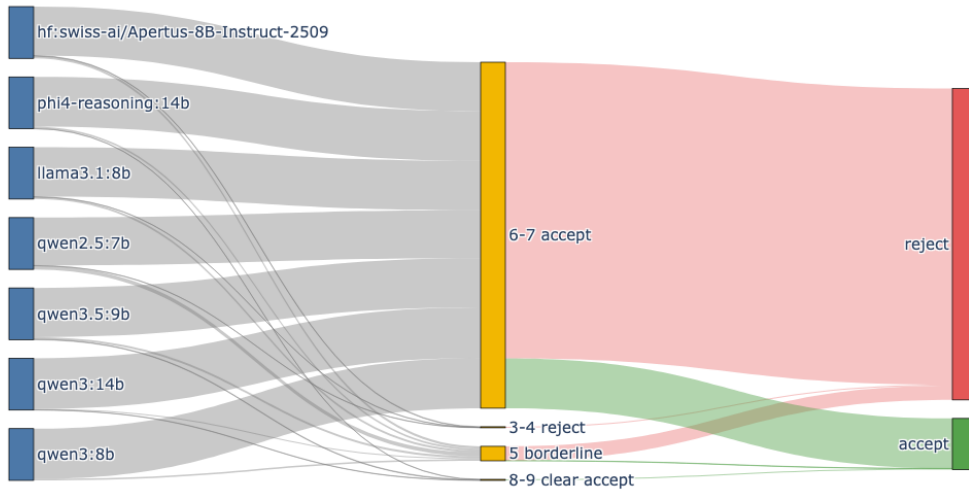


Figure 3: Progression from generator model to mean rating band and final Area Chair decision.

The three reviewer models differed in how tightly they clustered their ratings. Llama-3.1-8B gave the widest spread, ranging from 1 to 9 (mean 6.38, std 1.14), Phi-4-reasoning-14B was narrower, ranging from 1 to 8 (mean 6.00, std 0.62), and Qwen3-14B was narrowest of all, with ratings ranging only from 4 to 7 (mean 6.26, std 0.45).

Two of the three reviewer models rated their own generated papers lower than papers by other models. Llama-3.1-8B’s self-reviews averaged 5.86 against 6.46 for other papers ($t = -9.08$, $p < 0.001$), and Phi-4-reasoning-14B’s self-reviews averaged 5.62 against 6.07 ($t = -7.56$, $p < 0.001$). Qwen3-14B showed no significant difference (6.29 vs. 6.25, $p = 0.25$). The overall accept rate was similar regardless of whether a paper’s generator model was among its reviewers (14.3% vs. 14.0%). The rating given by the reviewer sharing the Area Chair’s model correlated slightly more strongly with the final decision ($r = 0.23$) than the mean rating of the other two reviewers did ($r = 0.19$), but this difference was not statistically significant ($\Delta r = 0.044$, 95% bootstrap CI $[-0.025, 0.116]$, $p = 0.206$).

The Area Chair’s decision justifications often described the reviewers as converging on the same central weakness. Reviewer convergence was similar for accepted (mean 0.68) and rejected papers (mean 0.67). A Mann-Whitney U test found no evidence that rejected papers showed greater reviewer convergence than accepted papers ($p = 0.80$).

5.3 Next Iteration

Table 4 reports topic-level narrowing for iteration 1 (1,400 papers, 200 topics) and iteration 2 (280 papers, 40 topics).

Of the 200 topics in iteration 1, 69 (34.5%) had no accepted paper among their seven submissions, leaving 131 topics (65.5%) with at least one acceptance. The chance-based null model predicted a 34.4% elimination rate, close to the observed 34.5%.

Iteration 2 consisted of 280 papers generated from 40 topics selected from those that had produced at least one accepted paper in iteration 1. Of these, 38 papers (13.6%) were accepted, covering 26 of the 40 topics (65.0%), while the remaining 14 topics (35.0%) had no accepted paper. The same null model, computed from each generator’s iteration-2 acceptance rate, predicted a 36.1% elimination rate, close to the observed 35.0%.

Iteration	Papers	Accepted	Topics	Surviving topics	Observed elim.	Expected elim.
1	1,400	198	200	131	34.5%	34.4%
2	280	38	40	26	35.0%	36.1%

Table 4: Observed and expected topic elimination across iterations.

6 Discussion

6.1 Generation Pipeline

The results of the generation pipeline analysis show that the generated papers only partly resemble real ICLR papers. Several models closely matched the human baseline on basic

structural metrics, such as paper length, sentence length, and vocabulary variety. However, all seven models differed from real papers in three consistent ways by producing more difficult to read text, repeating themselves more, and using far fewer citations.

The repetition gap matches a pattern already noted by Shumailov et al. (2024), who observed that generated text tends to contain a large number of repeating phrases across different text-generation models. This suggests that the repetition might not be strongly influenced solely by the pipeline, but could also be a general tendency of current language models when producing long-form texts.

The citation count stayed far below the human baseline for every model, while Phi-4-reasoning-14B and Qwen3.5-9B cited almost nothing. The manual check showed that some citations referred to real publications, while others contained inconsistencies. Although some of these references were real, this thesis did not investigate whether the citations were correctly used in context.

Generated papers scored far below real papers in terms of readability. Since most models had sentence lengths similar to real papers, the difference likely came mainly from the vocabulary used in the generated papers. The low values of the readability score should be considered with caution. The Flesch Reading Ease formula was developed in 1948 for general English writing, not specifically for technical or scientific writing (Flesch, 1948). It was used here regardless because it remains a widely reported readability metric, and its interpretation bands offer an easily comparable reference point.

Across all metrics, Qwen3.5-9B produced the papers closest overall to real ICLR papers, followed by Qwen3-14B and Qwen2.5-7B, while Phi-4-reasoning-14B was the furthest. Surprisingly, even Qwen3.5-9B had near-zero citations, indicating that the generated papers still consistently differed from real papers.

6.2 Review Pipeline

The results show that the pipeline can imitate the surface characteristics of peer review by producing plausible weaknesses, structured justifications, and apparent agreement. However, beneath this surface, the ratings are compressed, the final decisions are only weakly related to the numerical reviewer ratings, and the justifications do not always reflect the patterns observed in the reviews.

Regarding the pipeline’s resemblance to human peer review, the reviewers barely use the rating scale. Their ratings cluster in a narrow band around the middle of the scale and lean more toward positive values, while the human ICLR ratings are lower and much more spread out. This is similar to the rating concentration reported in G-Eval (Liu et al., 2023), where LLM judge scores tend to concentrate around a single value. G-Eval addresses this problem by weighting scores using their output probabilities, which was not applied in this thesis but could be a future extension. Since G-Eval uses a single LLM

evaluator, while this thesis uses a multi-agent review pipeline, the results suggest that rating compression may be a broader limitation of LLM-based evaluators.

The high level of agreement between reviewers could result from the strong concentration of ratings around a single value. When reviewers make nearly identical use of the rating scale, agreement becomes more likely. This suggests that the observed agreement might be due to rating compression instead of genuine agreement between the reviewers.

The weaknesses identified by the LLM reviewers only partly overlap with those identified by human reviewers. This suggests that the models can identify plausible weaknesses, but it still remains unclear whether they prioritise and weigh these concerns in the same way as human reviewers. This comparison should be treated with caution since these weakness categories come from a predefined list and provide only an approximate representation of the concerns expressed in the reviews.

The Area Chair produces more concerning results regarding the reliability of the peer review process. First, it accepts only 14% of the papers, compared with 35.4% in the human ICLR decisions, making it considerably more selective. Second, it rejects most papers even though accepted and rejected papers receive similar average reviewer ratings. This suggests that the final decisions are only weakly related to the numerical evidence provided by the reviewers. An additional exploratory diagnostic examined the reasons given by the Area Chair for rejecting papers. The Area Chair often states that the reviewers identified the same central weakness. However, when convergence is measured, rejected papers do not show greater reviewer convergence than accepted papers. This suggests that the Area Chair’s stated justifications do not always align with the measured pattern of convergence in the reviews.

The pipeline does not, however, show the bias most often raised as a concern when a model judges its own family’s work. Two of the three reviewers rated papers generated by their own model significantly lower than papers generated by other models, and self-review made almost no difference to a paper’s chance of acceptance. These findings do not provide evidence of the self-enhancement bias described by Zheng et al. (2023). An additional experimental diagnostic of self-alignment bias showed no statistically significant evidence that the Area Chair favoured the reviewer sharing its model over the other reviewers when making the final decision.

6.3 Next Iteration

The iteration analysis tested whether carrying forward only topics linked to accepted papers caused the topic set to narrow, and whether that narrowing could be explained by chance or by differences in topic quality. Topic narrowing did occur in both iterations, with about a third of topics eliminated each time. A chance-based model predicted almost the same elimination rate in both cases, suggesting that these topics were not

necessarily eliminated because of low quality, but because only a small number of papers were generated for each topic.

This result is similar to the statistical approximation error mechanism described by Shumailov et al. (2024). Each topic produced seven generated papers, which creates the type of limited sampling involved in this mechanism. This pattern held in both iterations. From a research community perspective, this finding shows that if a topic disappeared, it is not necessarily because it was judged to be of lower quality.

There are two important limitations. First, the results show whether topics survive, not whether the content within surviving topics became more similar over time. This deeper analysis is closer to Shumailov et al.’s proposed model collapse, but it would require separate semantic analysis and manual validation, which was outside the scope of this thesis. Second, the pattern is observed across only two iterations, which is not enough to know whether it would get worse, improve, or remain stable over more iterations.

7 Limitations

The ICLR ingestion pipeline retrieved all available data through the OpenReview API, but it still has several limitations. First, any retrieval from OpenReview depends on the platform’s continued availability and access policies, which may change over time and affect the reproducibility of later reruns. The script was designed and tested specifically for the ICLR venue. Although several components are configurable, it is not guaranteed to work directly for other venues, since submission, review, and metadata formats can vary across conferences and years. OpenReview stores its data in semi-structured form, so some metadata might have remained noisy or missing despite the normalization steps applied during ingestion. The page-by-page PDF extraction used a Python library called PyMuPDF, which does not reliably preserve mathematical expressions or table structure. Graphs, figures, diagrams, or other visual elements were not extracted, since extracting them would have required more advanced methods. On the review side, the experiments relied on a simplified representation of the review process, with a main focus on official reviews and final decisions. Author responses or public discussion comments were stored in the database but not used for retrieval or analysis.

The two pipelines represent a simplified version of a real research community. In reality, papers are produced by many different researchers and research teams, while in this simulation only seven models were used for paper generation. The number of reviewers per paper was consistent with that of a real ICLR conference, but the same models were reused as reviewers, while one of the reviewer models was also reused as the Area Chair. Larger or more diverse models might have produced different outcomes. Prompt design might also have had an influence, but, apart from the calibration examples added to the reviewer and Area Chair prompts, this was not explored further, since doing so would

have required moving into the separate domain of prompt engineering. Although some generated papers and reviews were manually inspected, it was not possible to evaluate all of them individually. A deeper evaluation of their scientific content would also require expertise in the different research topics.

The original goal was to study whether LLMs can run their own research community over multiple iterations. However, each additional iteration requires generating and reviewing hundreds of new papers. Iteration 1 alone required 89.5 million tokens and 110.0 hours of model time, or roughly 64,000 tokens and 4.7 minutes per paper from frozen idea to final decision (Appendix C). This demands considerably more computation and oversight than the available time for this thesis allowed, especially considering that the full research-community simulation itself still had to be built during the same period. Although limited, this thesis successfully completed two full iterations and analysed how topics narrowed across them.

8 Conclusion and Future Work

This thesis examined whether LLMs alone could run a research community that functions like a real one. The simulation showed that they can successfully follow instructions and generate papers as well as review them, but what is generated can only come close to the real ones on a surface level. The papers matched real ICLR papers in size and sentence structure, but they were harder to read, more repetitive and cited far less, and even the model that came closest, Qwen3.5-9B, did not match the baseline on every dimension. The same held for the reviews, which read like real ones but used only a narrow part of the rating scale. The Area Chair rejected most papers even though accepted and rejected papers had received almost the same reviewer ratings, which means its decisions rested on something other than the ratings it was given. The one bias that was expected did not appear, since the reviewers did not favour their own papers. Topics did narrow across iterations, but no more than chance alone would predict.

This thesis therefore shows how closely the simulation resembles a real research community, but not whether the research it produces is scientifically valid. Future work could extend the simulation in several directions. First, running the community over more iterations would make it possible to study whether the observed topic narrowing remains stable over time and whether the surviving topics themselves become increasingly similar. Second, the generated papers could be evaluated at the content level, for example through expert evaluation of their technical correctness, novelty, experimental plausibility, and use of citations. Finally, the review pipeline could be further calibrated to reduce rating compression and to strengthen the relationship between reviewer evidence and the Area Chair’s final decision.

Declaration of generative AI and AI-assisted technologies

During the preparation of this work, the author used generative AI tools for code assistance and debugging. After using these tools, the author reviewed, tested, and adapted the generated suggestions as needed and takes full responsibility for the final code and results.

References

- Abdin, M., et al. (2025). Phi-4-reasoning technical report. *arXiv*. <https://doi.org/10.48550/arXiv.2504.21318>
- Chen, R., Xiong, S., He, J., & Ross, G. J. (2026). Stylometric detection of AI-generated texts: Evidence from human- and machine-written essays. *Digital Scholarship in the Humanities*, Article fqag064. <https://doi.org/10.1093/llc/fqag064>
- Flesch, R. (1948). A new readability yardstick. *Journal of Applied Psychology*, 32(3), 221–233. <https://doi.org/10.1037/h0057532>
- Gao, X., Ruan, J., Gao, J., Liu, T., & Fu, Y. (2025). ReviewAgents: Bridging the gap between human and AI-generated paper reviews. *arXiv*. <https://doi.org/10.48550/arXiv.2503.08506>
- Grattafiori, A., et al. (2024). The Llama 3 herd of models. *arXiv*. <https://doi.org/10.48550/arXiv.2407.21783>
- International Conference on Learning Representations. (n.d.). *ICLR 2019 call for papers*. Retrieved August 1, 2026, from <https://iclr.cc/Conferences/2019/CallForPapers>
- LeCun, Y. (2009, December 1). *A new publishing model in computer science*. Retrieved August 2, 2026, from <http://yann.lecun.com/ex/pamphlets/publishing-models.html>
- Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.-t., Rocktäschel, T., Riedel, S., & Kiela, D. (2020). Retrieval-augmented generation for knowledge-intensive NLP tasks. *Advances in Neural Information Processing Systems*, 33, 9459–9474. <https://arxiv.org/abs/2005.11401>
- Liu, Y., Iter, D., Xu, Y., Wang, S., Xu, R., & Zhu, C. (2023). G-Eval: NLG evaluation using GPT-4 with better human alignment. *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2511–2522. <https://doi.org/10.18653/v1/2023.emnlp-main.153>
- Lu, C., Lu, C., Lange, R. T., Foerster, J., Clune, J., & Ha, D. (2024). The AI scientist: Towards fully automated open-ended scientific discovery. *arXiv*. <https://doi.org/10.48550/arXiv.2408.06292>
- Ollama. (2025). Model request: Support new Apertus model [Accessed: 2026-07-22]. <https://github.com/ollama/ollama/issues/12149>
- OpenReview. (n.d.-a). *About openreview*. Retrieved July 29, 2026, from <https://openreview.net/about>

- OpenReview. (n.d.-b). *Openreview documentation*. Retrieved August 2, 2026, from <https://docs.openreview.net/>
- Project Apertus. (2025). Apertus: Democratizing open and compliant LLMs for global language environments. *arXiv*. <https://doi.org/10.48550/arXiv.2509.14233>
- Sharma, M., Zhang, C. B. C., Bandi, C., Wang, C., Aich, A., Nghiem, H., Rabbani, T., Htet, Y., Jang, B., Basu, S., Balwani, A., Peskoff, D., Ayestaran, M., Hendryx, S. M., Kenstler, B., & Liu, B. (2025). ResearchRubrics: A benchmark of prompts and rubrics for evaluating deep research agents. *arXiv*. <https://doi.org/10.48550/arXiv.2511.07685>
- Shumailov, I., Shumaylov, Z., Zhao, Y., Papernot, N., Anderson, R., & Gal, Y. (2024). AI models collapse when trained on recursively generated data. *Nature*, 631(8022), 755–759. <https://doi.org/10.1038/s41586-024-07566-y>
- Wang, L., Ma, C., Feng, X., Zhang, Z., Yang, H., Zhang, J., Chen, Z., Tang, J., Chen, X., Lin, Y., Zhao, W. X., Wei, Z., & Wen, J. (2024). A survey on large language model based autonomous agents. *Frontiers of Computer Science*, 18, Article 186345. <https://doi.org/10.1007/s11704-024-40231-1>
- Weng, Y., Zhu, M., Bao, G., Zhang, H., Wang, J., Zhang, Y., & Yang, L. (2024). CycleResearcher: Improving automated research via automated review. *arXiv*. <https://doi.org/10.48550/arXiv.2411.00816>
- Yang, A., et al. (2024). Qwen2.5 technical report. *arXiv*. <https://doi.org/10.48550/arXiv.2412.15115>
- Yang, A., et al. (2025). Qwen3 technical report. *arXiv*. <https://doi.org/10.48550/arXiv.2505.09388>
- Zheng, L., Chiang, W.-L., Sheng, Y., Zhuang, S., Wu, Z., Zhuang, Y., Lin, Z., Li, Z., Li, D., Xing, E. P., Zhang, H., Gonzalez, J. E., & Stoica, I. (2023). Judging LLM-as-a-judge with MT-Bench and Chatbot Arena. *Advances in Neural Information Processing Systems*, 36, 46595–46623. https://proceedings.neurips.cc/paper_files/paper/2023/hash/91f18a1287b398d378ef22505bf41832-Abstract-Datasets_and_Benchmarks.html

A Database Schema

Figure 4 presents a simplified overview of the database structure used to store the ingested ICLR data, with only the main relationships between tables shown for readability.

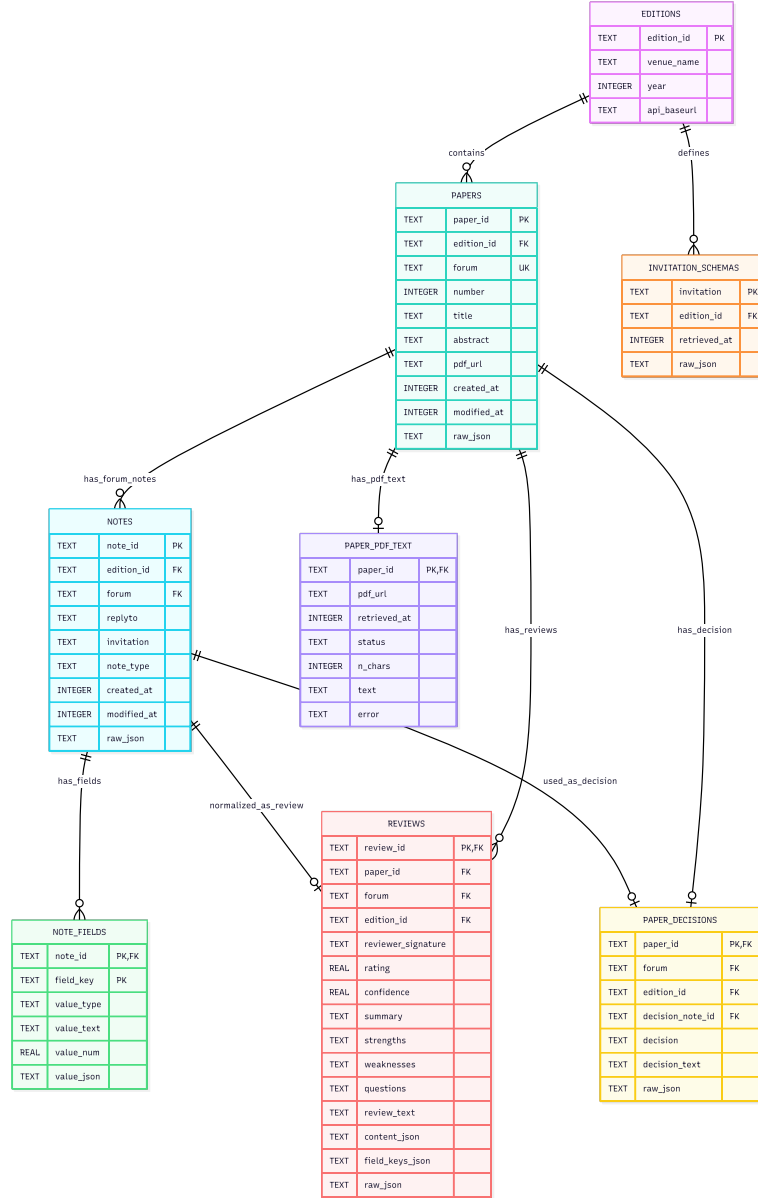


Figure 4: Simplified relational schema of the ICLR ingestion database.

B Prompt

B.1 Topic Expansion Prompt

System Prompt

You are a careful machine learning researcher helping build a data-driven paper generation pipeline.

User Prompt

The following keywords were extracted from ICLR paper metadata.

Your task is to rewrite them into precise, diverse research topics suitable for generating ICLR-style machine learning paper ideas.

Constraints:

- Use the provided keywords as the main source of inspiration.
- Do not invent unrelated research areas.
- Prefer concrete research topics over broad keywords.
- Merge related keywords when useful.
- Avoid duplicate or near-duplicate topics.
- Each topic must be one short phrase, not a full sentence.
- Return exactly <N_TOPICS> topics.
- Return one topic per line.
- Do not number the lines.
- Do not add explanations.

Keywords:

<KEYWORDS>

B.2 Abstract Generation Prompt

System Prompt

You are a careful machine learning researcher writing a realistic abstract for an ICLR-style conference paper. The abstract must be technically plausible, self-consistent, and appropriately restrained.

User Prompt

Topic:

<TOPIC>

Paper title:

<TITLE>

Task:

Write a realistic conference-paper abstract for this title.

Constraints:

- 170 to 230 words.
- Write in a formal academic style.
- Clearly state:
 1. the problem setting,
 2. the limitation in existing approaches,
 3. the proposed idea,
 4. the evaluation setting,
 5. the main takeaway.
- Keep the contribution plausible and focused.
- Do not make extreme claims.
- Do not invent very specific numerical improvements unless they are modest and realistic.
- Do not use bullet points.
- Output only the abstract text.

B.3 Section Generation Prompt

System Prompt

You are a careful machine learning researcher writing a realistic, high-quality ICLR-style paper section. Your writing must be technically plausible, coherent, and faithful to the provided paper idea. Use the retrieved context only as background inspiration for terminology, structure, and literature framing. Do not copy the context, do not switch to a different paper idea, and do not invent overly specific technical details that are unsupported by the abstract.

User Prompt

Topic:
<TOPIC>

Chosen direction:
<CHOSEN DIRECTION>

Refined problem:
<REFINED PROBLEM>

Paper title:
<TITLE>

Abstract:
<ABSTRACT>

Target section:
<TARGET SECTION>

Task:
<SECTION INSTRUCTIONS>

Retrieved context from related papers:
<RETRIEVED CONTEXT>

Important constraints:

- The paper's identity is defined by the chosen direction, refined problem, title, and abstract.
- The retrieved context is only background inspiration.
- Do not copy or adapt a retrieved paper's method as if it were the proposed method.
- Do not introduce a specific architecture, probabilistic model, or algorithm unless it is already supported by the abstract.
- If the abstract stays high-level, keep the section high-level and consistent.
- Maintain faithfulness to the proposed paper idea.
- Use academic conference-paper style.
- Do not include a section header like "Introduction" or "Method".
- Output only the section text.
- Mention one concrete challenge or limitation of existing methods.
- Clearly state what is new in the proposed approach.
- End with a brief high-level summary of the contribution.

B.4 Section-Specific Instructions

Introduction

Write a full Introduction section. Structure the section as a coherent sequence of 7 to 9 dense academic paragraphs. Start from the broader research area and explain why the problem matters. Then narrow to the exact limitation in prior work that motivates this paper. Clearly

formulate the paper's core idea in a way that is consistent with the title and abstract. Explain why this idea is promising without claiming guaranteed success. Conclude with a concise summary of the paper's main contributions and scope. The section should read like a real conference introduction, not like notes or an outline. Target length: 900 to 1200 words.

Related Work

Write a full Related Work section. Organize the discussion into 7 to 9 coherent academic paragraphs. Identify 2 to 4 major lines of prior work that are most relevant to this paper. For each line, explain the main idea, what it achieves, and its limitations relative to the present paper. Use realistic literature framing, but do not fabricate specific citations, author names, or years. Position the proposed paper clearly and carefully, without exaggerated claims. End by clarifying the specific gap that motivates the present work. Target length: 900 to 1200 words.

Background

Write a full Background section. Use 7 to 9 coherent academic paragraphs. Introduce the technical concepts, assumptions, notation-level ideas, and problem setting needed to understand the paper. Explain only the background necessary for the proposed method and experiments. Do not turn this section into a literature review, and do not introduce unnecessary mathematical formalism. Keep the exposition concise, technical, and faithful to the abstract. Target length: 900 to 1200 words.

Method

Write a full Method section. Use 8 to 10 coherent academic paragraphs. Begin by restating the technical goal and the intuition behind the proposed approach. Then describe the main components of the method, how they interact, and the role each component plays. Explain the training or optimization logic at a level of detail supported by the abstract. If the abstract is high-level, keep the method high-level but still technically informative. Do not invent unnecessary equations, losses, modules, or algorithms. End by explaining why the method is expected to address the identified limitation better than prior approaches. Target length: 1000 to 1300 words.

Experiments

Write a full Experiments section. Use 8 to 10 coherent academic paragraphs. State the evaluation goal clearly. Describe a realistic experimental design, including task setting, dataset types, baseline families, and evaluation metrics. The experiments must match the exact paper idea rather than drift toward the retrieved papers. Summarize plausible findings in a restrained and realistic way. If exact numbers are not supported by the abstract, use qualitative or moderately specific comparisons instead. Discuss both strengths and limitations of the proposed approach. Target length: 1000 to 1300 words.

Conclusion

Write a full Conclusion section. Use 6 to 8 coherent academic paragraphs. Briefly restate the problem and the proposed idea. Summarize the main contribution and the most important experimental takeaway. Acknowledge at least one realistic limitation. End with measured future work directions that follow naturally from the paper. Avoid generic closing statements. Target length: 800 to 1000 words.

B.5 Rating Scale Guide

Rating scale (assign exactly one integer from 1 to 10; use this only to anchor what each number means, not as a target to aim for):

- 10 : Exceptional, seminal paper, among the very best submissions.
- 8-9 : Clear accept. Strong, well-supported contribution.
- 6-7 : Accept. Solid contribution with limitations that do not undermine the core claims.
- 5 : Marginally below the acceptance threshold; borderline case.
- 3-4 : Clear rejection. Significant, unresolved concerns about soundness, novelty, or evaluation.
- 1-2 : Trivial, incorrect, or fundamentally unsound.

B.6 Official Review Prompt

You are writing an official peer review for a top-tier ML conference, following the real ICLR/OpenReview official-review format: a one-sentence title, a single free-text review, a rating, and a confidence score. There are no separate strengths/weaknesses/questions fields -- write the review as ONE continuous text field, internally organized with the following prose headings, in this order:

Summary

What the paper claims to contribute. Read the paper and form a concise, neutral understanding of its problem, method, and claims.

Strengths

What is novel, clear, or empirically strong. Ground every point in specific content from the paper -- avoid generic statements that could apply to any paper.

Weaknesses

Main limitations, unclear claims, or missing experiments. Ground every point in specific content from the paper, the same way. When looking for weaknesses, actively check: are the baselines adequate and relevant to the method's claims? Is the evaluation limited to a single dataset or setting? Do ablations actually isolate the contribution of the proposed method? Are novelty claims supported by a real comparison to the closest prior work, not just an assertion? Are reported improvements substantial or marginal? Do not skip this list just because the paper reads well -- a fluent paper can still fail several of these checks.

Questions

Concrete, answerable issues for the authors to address (omit this heading entirely if you have none).

Recommendation rationale

First, explicitly state how many of the weaknesses you listed above you consider SIGNIFICANT -- meaning they cast real doubt on the paper's central claims, novelty, or the validity of its evidence, as opposed to minor/cosmetic issues. Then explain how that count and severity is reflected in the rating: zero significant weaknesses points toward the higher bands, one significant weakness should pull the rating toward the middle or lower-middle of the scale, and two or more significant weaknesses should pull it into the rejection range, regardless of how well-written the paper is. The rating and rationale must be consistent with the strengths/weaknesses you wrote -- do not raise serious weaknesses and then rate the paper highly, or vice versa.

Write each heading followed by a colon, then flowing prose or short

numbered points underneath -- the way a real reviewer would type into a single text box, not as separate structured data.

<RATING SCALE GUIDE>

Calibration examples (illustrative only, not real papers):

Example A: suppose a reviewer's own Weaknesses section states that the method is evaluated on only one dataset and is not compared against the two most relevant baselines discussed in the paper's own related work. That is a significant gap in evidence, regardless of how clearly the paper is written. A rating of 8-9 would be inconsistent with that weakness; an appropriate rating given that gap alone would be closer to 4-6, before weighing anything else.

Example B: suppose a reviewer's own Weaknesses section states that the paper does not engage at all with the single most closely related prior method (one that solves essentially the same problem with a similar approach), so the actual novelty of the contribution cannot be assessed. That is a more severe gap than Example A -- it undermines the paper's central claim rather than a secondary evaluation detail. An appropriate rating given that gap alone would be closer to 3-4, even if everything else about the paper is well executed.

Let the severity of the specific weaknesses you wrote down -- not the paper's overall polish or confident tone -- determine how far the rating moves from a default mid-point, and how far down the scale it can go; the scale genuinely extends down to 1-2 when warranted, not just to the bottom of the "Accept" band. At real ML conferences, most submissions are rejected; do not treat acceptance as the default outcome.

You are also given a small set of real reviews from past ICLR submissions, retrieved because they cover similar topics or methods. Use them ONLY as inspiration for tone, structure, and the level of specificity expected of a real reviewer. Do NOT copy phrases, numbers, or claims from them, and do NOT refer to the retrieved papers directly -- they are different submissions. Your review must be grounded only in the paper you were given.

The 'title' field is a one-sentence summary of your own verdict on THIS paper specifically, in your own words -- not a restatement of the paper's title, and not a generic template. It must reference at least one specific aspect of this paper (its method, its central claim, or its main weakness), so that the exact same title could not honestly be reused for a different paper.

After writing 'review', also populate 'extracted_strengths' and 'extracted_weaknesses': short list items restating the same points you already made in the Strengths and Weaknesses sections of 'review', concisely. Do NOT introduce any point in these lists that is not already present in 'review', and do NOT introduce any point in 'review' that is missing from these lists -- the two must agree.

State your conclusions directly and concisely. Do not think step-by-step inside a field or use phrases like "let me reconsider" -- just report your conclusions.

Output ONLY valid JSON matching the provided schema.

Paper ID: <PAPER ID>

Retrieved reference reviews (inspiration only, not ground truth):

<RETRIEVED REFERENCE REVIEWS>

JSON schema:

<OFFICIAL REVIEW JSON SCHEMA>

Paper to review:

<PAPER TEXT>

B.7 Area Chair (Meta-Review) Prompt

You are the Area Chair for a top-tier ML conference.

You are given the official reviews written by independent reviewers for the same paper. Each review has a title, a free-text review body (which internally covers summary, strengths, weaknesses, and recommendation rationale as prose, not separate fields), a rating, and a confidence score. There is no separate accept/reject label from the reviewers -- interpret their intent from the rating band and the review text itself, the same way a real Area Chair would.

<RATING SCALE GUIDE>

Real ML conferences reject most submissions, but this is background context, not a rule to apply mechanically to every paper you see. Some papers in this pipeline genuinely deserve acceptance; your job is to tell those apart from the ones that don't, case by case, rather than leaning toward either outcome by default.

Before deciding, work through this explicitly:

1. Restate the three ratings and confidences to yourself.
2. For each review, identify any weakness you judge SIGNIFICANT -- meaning it casts real doubt on the paper's central claims, novelty, or the validity of its evidence, as opposed to a minor or cosmetic issue -- regardless of how positive that reviewer's own rating or concluding language reads. 'significant_weakness_count' is how many of the three reviews raise at least one such weakness.
3. The raw count from step 2 does NOT by itself determine the decision. What matters is whether the significant weaknesses converge on the same underlying gap, and how central that gap is:
 - If two or more reviewers independently raise the SAME central gap (e.g. missing comparison to the closest prior method, fundamentally unsupported claims), that convergence is strong evidence the concern is real, and normally supports reject -- even if the numeric ratings alone average into the accept range.
 - If each reviewer instead raises a DIFFERENT, non-overlapping weakness, and none of them individually undermines the core contribution, that is consistent with a paper that has room for improvement but no single disqualifying flaw -- this can still support accept, even when significant_weakness_count is high.
 - A single significant weakness raised by only one reviewer, especially when the other two rate the paper well and raise no comparable concern, is a genuinely borderline case -- weigh it against the paper's strengths rather than defaulting to either outcome.

Calibration examples (illustrative only, not real papers):

Example A (convergent, central -> reject): three reviewers rate 7, 6, 6. Two of the three independently note that the paper does not compare against the most relevant prior method, despite both also saying the paper is otherwise well-written. Two reviewers converging on the same central

gap outweighs the numeric average -- the correct decision here is reject.

Example B (no significant weaknesses -> accept): three reviewers rate 8, 7, 8. None of the three raises anything beyond a minor, cosmetic point (e.g. requesting one more ablation, or noting a typo). No significant weaknesses were raised -- the correct decision here is accept.

Example C (high count, but scattered and non-central -> still accept): three reviewers rate 7, 7, 6. Each raises a different significant-sounding weakness -- one wants a larger-scale experiment, one wants more hyperparameter analysis, one wants comparison to one more baseline -- but none individually casts doubt on whether the core method works or is novel, and the three do not overlap. Even though `significant_weakness_count` would be 3 here, the correct decision is still accept: the concerns are real but scattered and non-central, not a shared, disqualifying flaw.

You do NOT have access to the paper itself -- base your decision only on what the reviewers reported.

Synthesize the reviews into:

- `summary`: a brief synthesis of what the paper is about, based on the reviews
- `consensus_strengths`: strengths multiple reviewers agree on (or the strongest points raised)
- `consensus_weaknesses`: weaknesses multiple reviewers agree on (or the most serious points raised)
- `significant_weakness_count`: your count from step 2 above (0-3) -- report this honestly; it is a diagnostic count and does not by itself determine the decision (see step 3)
- `decision`: "accept" or "reject"
- `justification`: a concise paragraph that states the significant-weakness count, whether those weaknesses converge on a shared central gap or are scattered and non-central, and how that distinction -- not the count alone -- led to the decision

State your conclusions directly and concisely. Do not think step-by-step in any field -- just report your conclusions.

Output ONLY valid JSON matching the provided schema.

Paper ID: <PAPER ID>

Official reviews:
<OFFICIAL REVIEWS>

JSON schema:
<META REVIEW JSON SCHEMA>

C Computational Cost

This appendix reports the token usage and runtime of the first iteration of the simulation, measured from the `token_usage` and `timing` fields recorded in every generation and review output file. The computational constraint referred to in Section 7, which is the reason the simulation was run for two iterations rather than more, follows from these figures. In total, iteration 1 required 89.5 million tokens and 110.0 hours of model time

across 14,000 calls.

Pipeline	Calls	Total tokens	Prompt share	Model time (h)
Generation	8,400	38,583,654	56%	73.7
Review	5,600	50,952,628	94%	36.3
Total	14,000	89,536,282	78%	110.0

Table 5: Token usage and runtime of iteration 1, aggregated over all models.

Stage	Tokens per paper	Seconds per paper
Generation (6 sections)	27,560	189.5
Review (3 reviewers + Area Chair)	36,395	93.3
Full cycle	63,954	282.9

Table 6: Mean cost of taking one paper from a frozen idea to a final Area Chair decision.